

Week 5 Stabilization Testing Report

Project Title

Stabilization Testing for SDET Retail POS Application

1. Introduction

Stabilization testing is performed to verify that an application remains reliable, consistent, and defect-free after multiple feature implementations and fixes. During Week 5, stabilization testing was conducted on the Retail Point of Sale (POS) application to validate critical business workflows under both normal and adverse operating conditions.

The primary focus of this testing cycle was to evaluate the application's behavior during network interruptions, synchronization recovery, refund processing, and diagnostic logging. Automated tests were developed and executed using Playwright to ensure repeatable and reliable validation of these scenarios.

2. Aim

The aim of this project was to validate the stability, reliability, and resilience of the Retail POS application during Week 5 stabilization testing. The testing effort focused on ensuring that key business operations continue to function correctly under real-world conditions, including offline operation, transaction synchronization, refund processing, and system diagnostics.

3. Objectives

The major objectives of this stabilization testing activity were:

- Verify that sales transactions can be successfully queued when the POS system is operating in offline mode.
- Validate that queued transactions synchronize correctly once network connectivity is restored.
- Ensure that refund workflows function accurately and maintain transaction integrity.
- Confirm that application status indicators provide correct information to users during offline and online transitions.

- Verify that diagnostic logs and evidence files capture meaningful information for troubleshooting and validation.
 - Ensure automated Playwright test cases execute consistently and provide reliable regression coverage.
 - Identify potential defects, synchronization issues, or workflow inconsistencies before deployment.
-

4. Scope of Testing

The stabilization testing scope included the following functional and non-functional areas:

Functional Testing

- Offline sales transaction processing
- Transaction queue management
- Network reconnection handling
- Automatic synchronization of queued transactions
- Refund processing workflows
- Status updates and user notifications

Validation Activities

- Data consistency checks
- Error handling validation
- Diagnostic evidence generation
- Logging verification

Automation Coverage

- Browser-based automation using Playwright
 - API response validation
 - UI workflow verification
 - Automated regression checks
-

5. Test Environment

The following environment was used during test execution:

Component	Details
Browser	Chromium
Automation Framework	Playwright
Programming Language	TypeScript
Application Environment	Local Test Environment
Execution Type	Automated UI and API Testing
Operating System	Windows
Reporting Tools	Playwright HTML Reports, Screenshots, Trace Files

6. Test Automation Summary

Automation scripts were developed using Playwright to validate the stabilization requirements.

Test Files Executed

1. POS Offline Testing

- tests/pos-offline.spec.ts
- Validates offline sales creation and queue behavior.

2. Refund Processing Testing

- tests/refunds.spec.ts
- Verifies refund creation, validation rules, and transaction outcomes.

3. Diagnostics and Logging Testing

- tests/diagnostics-logging.spec.ts
- Ensures logs and diagnostic evidence are generated correctly.

Automation Benefits

- Faster execution of repetitive test cases.
- Consistent validation across multiple runs.
- Early detection of regression issues.

- Reliable evidence collection for audit and debugging purposes.
-

7. Test Execution Details

Test Execution Command

npm test

Execution Approach

The automated suite was executed against the Chromium browser. Tests simulated various operational scenarios including network failures, transaction queueing, synchronization recovery, refund validation, and diagnostic logging.

The execution process included:

- Launching the POS application.
 - Simulating offline network conditions.
 - Creating and validating queued sales.
 - Restoring connectivity.
 - Verifying synchronization behavior.
 - Executing refund scenarios.
 - Capturing diagnostic evidence and logs.
 - Generating Playwright reports and screenshots.
-

8. Test Run Results

Execution Summary

Metric	Result
Total Tests Executed	3
Passed	3
Failed	0

9. Detailed Observations

During stabilization testing, the following observations were recorded:

Offline Sale Queue Validation

- Sales transactions were successfully created while the application was offline.
- Queued transactions remained visible in the outbox until synchronization occurred.
- No transaction loss was observed during offline operation.

Synchronization Validation

- Upon restoring network connectivity, queued transactions synchronized automatically.
- Synchronization status indicators updated correctly.
- Transaction data remained consistent before and after synchronization.

Refund Processing Validation

- Refund requests were processed according to expected business rules.
- Transaction values and statuses were updated accurately.
- No duplicate refund processing was observed.

Diagnostic and Logging Validation

- Application logs were generated successfully.
- Diagnostic evidence was available for troubleshooting.
- Playwright traces and screenshots provided useful execution details.

User Interface Validation

- Offline mode indicators displayed correctly.
- Synchronization notifications were visible.
- Transaction statuses updated appropriately throughout the workflow.

10. Screenshots and Evidence Attachment

The following evidence should be attached to support the test execution results:

Offline Pos sales Queue

Q Search tests

All 3Passed 3Failed 0Flaky 0Skipped 0

Week 5 POS stabilization

offline POS sale queues while offline, reconnects once, and publishes evidence

pos-offline.spec.ts:9

View Trace | 5.4s

chromium

Run

Test Steps

Traces



trace

Attachments

Filter steps

> ✓ Before Hooks	2.0s
> ✓ GET "/api/sales" — pos-offline.spec.ts:19	134ms
> ✓ Expect "toBe" — pos-offline.spec.ts:22	1ms
> ✓ Attach "sales-before.json" — ./fixtures/test.ts:72	4ms
> ✓ Navigate to "/pos" — pos-offline.spec.ts:41	1.4s
> ✓ Click getByRole('button', { name: 'Reset lab' }) — pos-offline.spec.ts:42	305ms
> ✓ Set offline mode — pos-offline.spec.ts:44	7ms
> ✓ Expect "toContainText" getByTestId('network-banner') — pos-offline.spec.ts:45	68ms
> ✓ Click getByRole('button', { name: 'Queue sale' }) — pos-offline.spec.ts:46	164ms
> ✓ Expect "toContainText" getByTestId('pos-sync-status') — pos-offline.spec.ts:47	57ms
> ✓ Expect "toHaveText" getByTestId('outbox-count') — pos-offline.spec.ts:48	32ms
> ✓ Evaluate — pos-offline.spec.ts:50	39ms
> ✓ Attach "offline-outbox.json" — ./fixtures/test.ts:72	3ms
> ✓ Expect "toHaveLength" — pos-offline.spec.ts:54	2ms
> ✓ Expect "toBe" — pos-offline.spec.ts:55	0ms
> ✓ Set offline mode — pos-offline.spec.ts:57	7ms
> ✓ Expect "toContainText" getByTestId('network-banner') — pos-offline.spec.ts:58	119ms

▼ Attachments

▼ sales-before.json

```
{
  "items": []
}
```

> offline-outbox.json

▼ sales-after.json

```
{
  "items": [
    {
      "id": 9004,
      "saleNumber": "SALE-9004",
      "clientSaleId": "sale-1783161152563-pjpore4bv6",
      "ownerKey": "gate-pos-1783161150457",
      "cashier": "customer@example.com",
      "productId": 101,
      "productName": "Running Shoes",
      "quantity": 1,
      "total": 4499,
      "status": "SYNCED",
      "syncedAt": "2026-07-04T10:32:32.881Z"
    }
  ]
}
```

> stdout

Diagnostics-logging

Project: chromium

4/7/2026, 4:12:26 pm Total time: 2.3m

▼ diagnostics-logging.spec.ts

✓ Week 5 diagnostics and logging › records a correlated cart diagnostic trail

chromium

8.5s

diagnostics-logging.spec.ts:9 View Trace

✓ Week 5 diagnostics and logging › records a correlated cart diagnostic trail

chromium

6.4s

diagnostics-logging.spec.ts:9 View Trace

✓ Week 5 diagnostics and logging › records a correlated cart diagnostic trail

chromium

7.3s

diagnostics-logging.spec.ts:9 View Trace

✓ Week 5 diagnostics and logging › records a correlated cart diagnostic trail

chromium

6.6s

diagnostics-logging.spec.ts:9 View Trace

✓ Week 5 diagnostics and logging › records a correlated cart diagnostic trail

chromium

6.7s

diagnostics-logging.spec.ts:9 View Trace

✓ Week 5 diagnostics and logging › records a correlated cart diagnostic trail

chromium

7.5s

diagnostics-logging.spec.ts:9 View Trace

✓ Week 5 diagnostics and logging › records a correlated cart diagnostic trail

chromium

7.0s

diagnostics-logging.spec.ts:9 View Trace

✓ Week 5 diagnostics and logging › records a correlated cart diagnostic trail

chromium

8.8s

diagnostics-logging.spec.ts:9 View Trace

✓ Week 5 diagnostics and logging › records a correlated cart diagnostic trail

chromium

3.8s

diagnostics-logging.spec.ts:9 View Trace

Expect › expect › expect › diagnostics-logging.spec.ts

> ✓ Expect "toBeGreaterThan" — diagnostics-logging.spec.ts:50

0ms

> ✓ After Hooks

4.3s

▼ Traces



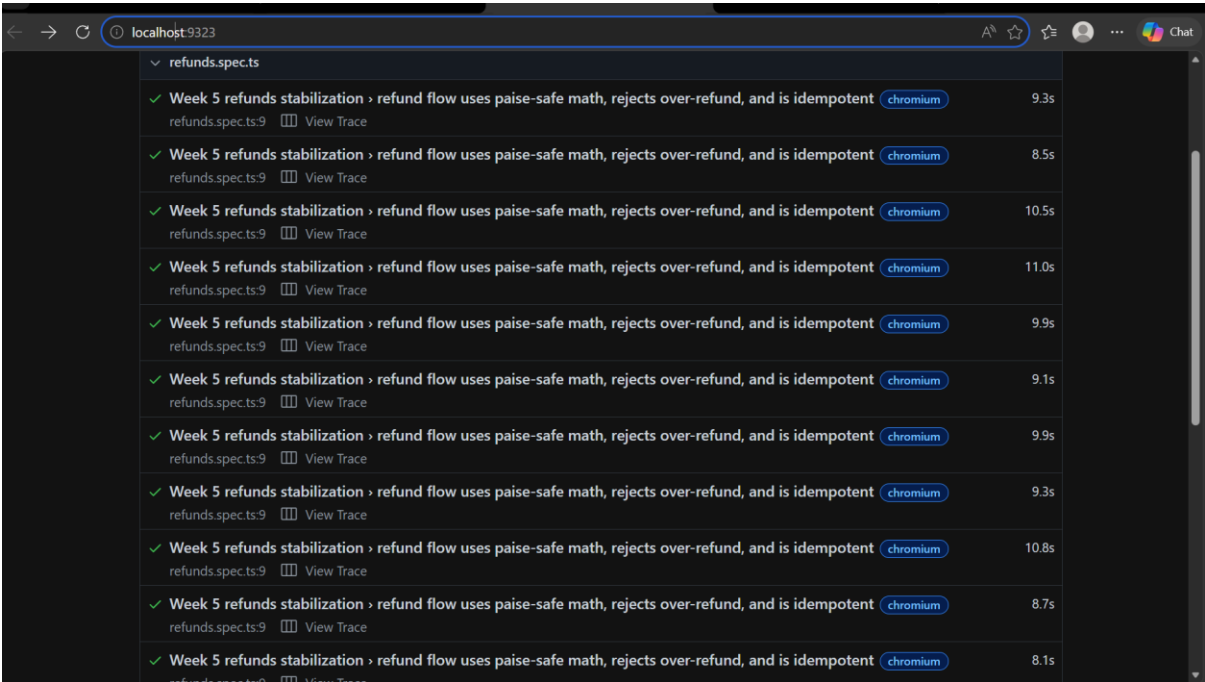
trace

▼ Attachments

> stdout

> Executed in Worker #0

refund



The screenshot shows a web browser window with the address bar set to 'localhost:9323'. The main content area displays a list of test results under the heading 'refunds.spec.ts'. Each row represents a test run, all of which are successful, indicated by a green checkmark. The test description for each row is 'Week 5 refunds stabilization > refund flow uses pause-safe math, rejects over-refund, and is idempotent'. The browser used for each test is 'chromium'. The execution time for each test is listed on the right side of the row. Below the test description, there is a link to 'View Trace'.

Test Description	Browser	Duration
Week 5 refunds stabilization > refund flow uses pause-safe math, rejects over-refund, and is idempotent	chromium	9.3s
Week 5 refunds stabilization > refund flow uses pause-safe math, rejects over-refund, and is idempotent	chromium	8.5s
Week 5 refunds stabilization > refund flow uses pause-safe math, rejects over-refund, and is idempotent	chromium	10.5s
Week 5 refunds stabilization > refund flow uses pause-safe math, rejects over-refund, and is idempotent	chromium	11.0s
Week 5 refunds stabilization > refund flow uses pause-safe math, rejects over-refund, and is idempotent	chromium	9.9s
Week 5 refunds stabilization > refund flow uses pause-safe math, rejects over-refund, and is idempotent	chromium	9.1s
Week 5 refunds stabilization > refund flow uses pause-safe math, rejects over-refund, and is idempotent	chromium	9.9s
Week 5 refunds stabilization > refund flow uses pause-safe math, rejects over-refund, and is idempotent	chromium	9.3s
Week 5 refunds stabilization > refund flow uses pause-safe math, rejects over-refund, and is idempotent	chromium	10.8s
Week 5 refunds stabilization > refund flow uses pause-safe math, rejects over-refund, and is idempotent	chromium	8.7s
Week 5 refunds stabilization > refund flow uses pause-safe math, rejects over-refund, and is idempotent	chromium	8.1s

11. Conclusion

The Week 5 stabilization testing cycle successfully validated critical Retail POS workflows under both normal and offline operating conditions. The application demonstrated stable behavior during transaction queueing, synchronization recovery, refund processing, and diagnostic logging activities.

The Playwright automation suite provided reliable regression coverage and enabled efficient validation of key business scenarios. Test evidence, logs, screenshots, and reports further supported the verification process and helped establish confidence in application quality.

Based on the executed test scenarios and observed results, the application shows a strong level of stability and readiness for subsequent testing phases and deployment activities.

supporting a smoother deployment process and a more reliable user experience.